

Lecture 12: Kernel Methods

Introduction to Machine Learning

Sajjad Amini

Department of Electrical Engineering
Sharif University of Technology

- 1 Approach Definition
- 2 Mercer Kernel
- 3 Gaussian Processes
- 4 Support Vector Machines
 - Hard Margin
 - Hard Margin SVM with Kernel
 - Soft Margin SVM
 - Soft Margin SVM with Kernel
 - Samples of SVM
 - Multiclass SVM

Except explicitly cited, the reference for the material in slides is:

- Murphy, K. P. (2022). *Probabilistic machine learning: an introduction*. MIT press.

Section 1

Approach Definition

Previous Methods

Till now, we use the following procedure to evaluate the output y given input vector $\mathbf{x}$:

- Considering a parameterized model with parameter vector θ
- Estimating parameters using dataset $\{(\mathbf{x}_n, y_n)\}$ ($\hat{\theta}$)
 - Plug-in approximation
 - Posterior distribution calculation
- Evaluating y using assumed model
 - Evaluating using plug-in approximation (1 model)
 - Evaluating using posterior predictive distribution (All possible models)

Approach Definition

Kernel Method

Assume we have dataset $\{(\mathbf{x}_n, y_n)\}$ and unknown function f that $y = f(\mathbf{x})$, then in kernel method, the procedure is:

- Evaluate a similarity between query vector $\mathbf{x}_q$ and all of input training vectors $\{\mathbf{x}_i\}_{i=1}^N$
- Use the measures similarity as weights to generate $f(\mathbf{x}_q)$ based on $\{f(\mathbf{x}_i)\}_{i=1}^N$

Note that *Kernels* are special functions that determine the similarity used for $f(\mathbf{x}_q)$ estimation.

Kernel Methods

- Kernel methods are nonparametric
- In model based methods, we compress the dataset information into a fixed length vector $\hat{\boldsymbol{\theta}}$, while in kernel method we need dataset to estimate $f(\mathbf{x}_q)$

Section 2

Mercer Kernel

Mercer Kernel [1]

Kernel

Assume an abstract space $\mathcal{X}$. Then function $\mathcal{K} : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is called a kernel function. Kernel function usually (not necessarily) have the following properties:

Symmetry: $\forall \mathbf{x}, \mathbf{x}' : \mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}(\mathbf{x}', \mathbf{x})$

Positivity: $\forall \mathbf{x}, \mathbf{x}' : \mathcal{K}(\mathbf{x}, \mathbf{x}') \geq 0, (\mathbf{x} \neq \mathbf{x}')$

Mercer (positive definite) Kernel

A symmetric kernel $\mathcal{K} : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^+$ is Mercer (PSD) kernel if:

$$\sum_{i=1}^N \sum_{j=1}^N \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) c_i c_j \geq 0$$

for any finite set of N distinct samples from $\mathcal{X} \supseteq X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and any vector $\mathbf{c} \in \mathbb{R}^N$.

Gram matrix for N Datapoints

Given N datapoints $X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$ and symmetric kernel $\mathcal{K} : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$, the Gram matrix is:

$$\mathbf{K} = \begin{bmatrix} \mathcal{K}(\mathbf{x}_1, \mathbf{x}_1) & \dots & \mathcal{K}(\mathbf{x}_1, \mathbf{x}_N) \\ \vdots & \vdots & \vdots \\ \mathcal{K}(\mathbf{x}_N, \mathbf{x}_1) & \dots & \mathcal{K}(\mathbf{x}_N, \mathbf{x}_N) \end{bmatrix}$$

Mercer Kernel and Gram Matrix

Symmetric positive kernel $\mathcal{K} : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ is Mercer iff the Gram matrix is positive definite for any finite set of N distinct samples from $\mathcal{X} \supseteq X = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$.

Hilbert Space

Inner Product

Let $\mathcal{H}$ be a vector space over $\mathbb{R}$. A function $\langle \cdot, \cdot \rangle_{\mathcal{H}} : \mathcal{H} \times \mathcal{H} \rightarrow \mathbb{R}$ is said to be an inner product on $\mathcal{H}$ if:

- ① $\langle \alpha_1 f_1 + \alpha_2 f_2, g \rangle_{\mathcal{H}} = \alpha_1 \langle f_1, g \rangle_{\mathcal{H}} + \alpha_2 \langle f_2, g \rangle_{\mathcal{H}}$
- ② $\langle f, g \rangle_{\mathcal{H}} = \langle g, f \rangle_{\mathcal{H}}$
- ③ $\langle f, f \rangle_{\mathcal{H}} \geq 0$ and $\langle f, f \rangle_{\mathcal{H}} = 0$ if and only if $f = 0$

Norm

Using inner product, we can define norm as: $\|f\|_{\mathcal{H}} = \sqrt{\langle f, f \rangle_{\mathcal{H}}}$

Hilbert Space

A Hilbert space is a vector space on which an inner product is defined (along with other technical conditions)

Checking Positive Definiteness of Kernel

Mercer (positive definite) Kernel

Let $\mathcal{H}$ be any Hilbert space, $\mathcal{X}$ a non-empty set and $\phi : \mathcal{X} \rightarrow \mathcal{H}$. Then kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle_{\mathcal{H}}$ is positive definite.

Proof:

$$\sum_{i=1}^N \sum_{j=1}^N \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) c_i c_j = \sum_{i=1}^N \sum_{j=1}^N \langle c_i \phi(\mathbf{x}_i), c_j \phi(\mathbf{x}_j) \rangle_{\mathcal{H}} = \left\| \sum_{i=1}^N c_i \phi(\mathbf{x}_i) \right\|_{\mathcal{H}}^2 \geq 0$$

Simple Mercer Kernel

Show that kernel $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$, $\mathbf{x} \in \mathbb{R}^n$ is Mercer.

Solution: We can introduce Hilbert space $\mathcal{H} = \mathbb{R}^n$ with the definition $\langle \mathbf{x}, \mathbf{x}' \rangle = \mathbf{x}^T \mathbf{x}'$ for inner product and mapping $\phi(\mathbf{x}) = \mathbf{x}$. Thus $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathbf{x}^T \mathbf{x}'$ is a Mercer kernel.

Mercer's Theorem

Intuition

Mercer's Theorem gives some idea on the space where the calculations are being done.

Theorem

Let $k(x, y)$ be a continuous, symmetric, and positive semi-definite function on a compact domain $D = [a, b] \times [a, b] \subset \mathbb{R}^2$. Then there exists a sequence of non-negative eigenvalues $\{\lambda_i\}_{i=1}^{\infty}$ and an orthonormal set of eigenfunctions $\{\psi_i\}_{i=1}^{\infty}$ in $L^2[a, b]$ such that:

$$k(x, y) = \sum_{i=1}^{\infty} \lambda_i \psi_i(x) \psi_i(y)$$

where the series converges absolutely and uniformly.

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$.

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$. Thus:

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$. Thus:

$$\mathbf{K} = \begin{bmatrix} | & & | \\ \hat{\mathbf{u}}_1 & \cdots & \hat{\mathbf{u}}_N \\ | & & | \end{bmatrix} \begin{bmatrix} - & \hat{\mathbf{u}}_1^T & - \\ & \vdots & \\ - & \hat{\mathbf{u}}_N^T & - \end{bmatrix}$$

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$. Thus:

$$\mathbf{K} = \begin{bmatrix} | & & | \\ \hat{\mathbf{u}}_1 & \cdots & \hat{\mathbf{u}}_N \\ | & & | \end{bmatrix} \begin{bmatrix} - & \hat{\mathbf{u}}_1^T & - \\ & \vdots & \\ - & \hat{\mathbf{u}}_N^T & - \end{bmatrix} \Rightarrow k_{ij} = \sum_{n=1}^N \hat{\mathbf{u}}_n(i) \hat{\mathbf{u}}_n(j)$$

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$. Thus:

$$\mathbf{K} = \begin{bmatrix} | & & | \\ \hat{\mathbf{u}}_1 & \cdots & \hat{\mathbf{u}}_N \\ | & & | \end{bmatrix} \begin{bmatrix} - & \hat{\mathbf{u}}_1^T & - \\ & \vdots & \\ - & \hat{\mathbf{u}}_N^T & - \end{bmatrix} \Rightarrow k_{ij} = \sum_{n=1}^N \hat{\mathbf{u}}_n(i) \hat{\mathbf{u}}_n(j) = \langle \tilde{\mathbf{u}}_i, \tilde{\mathbf{u}}_j \rangle$$

Mercer's Theorem

Simple Case

Assume Gram matrix $\mathbf{K}$ to be positive definite. Then from eigen decomposition we have $\mathbf{K} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$ where:

$$\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_N), \lambda_i > 0 \text{ for } i = 1, \dots, N$$

$$\mathbf{U} = [\mathbf{u}_1, \dots, \mathbf{u}_N]$$

We can rewrite $(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})(\mathbf{U}\mathbf{\Lambda}^{\frac{1}{2}})^T = \hat{\mathbf{U}}\hat{\mathbf{U}}^T$ where $\hat{\mathbf{U}} = [\hat{\mathbf{u}}_1, \dots, \hat{\mathbf{u}}_N]$. Thus:

$$\mathbf{K} = \begin{bmatrix} | & & | \\ \hat{\mathbf{u}}_1 & \cdots & \hat{\mathbf{u}}_N \\ | & & | \end{bmatrix} \begin{bmatrix} - & \hat{\mathbf{u}}_1^T & - \\ & \vdots & \\ - & \hat{\mathbf{u}}_N^T & - \end{bmatrix} \Rightarrow k_{ij} = \sum_{n=1}^N \hat{\mathbf{u}}_n(i) \hat{\mathbf{u}}_n(j) = \langle \tilde{\mathbf{u}}_i, \tilde{\mathbf{u}}_j \rangle$$

So $\phi(\mathbf{x}_i) = \tilde{\mathbf{u}}_i$ and we can write the entries in form of inner product.

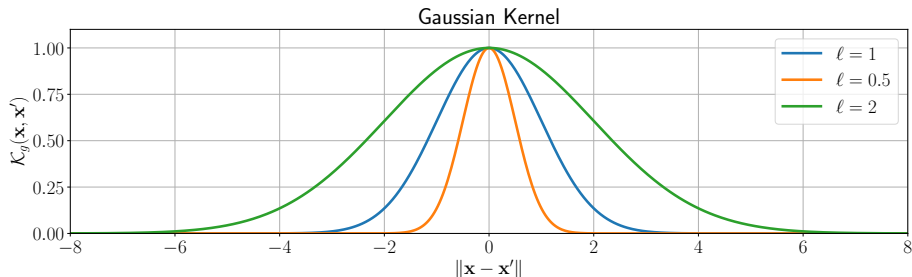
Popular Mercer Kernel [2]

Stationary Kernels

Kernels of the form $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \mathcal{K}(\|\mathbf{x} - \mathbf{x}'\|)$ are called stationary kernels.

Gaussian (exponentiated quadratic) kernel

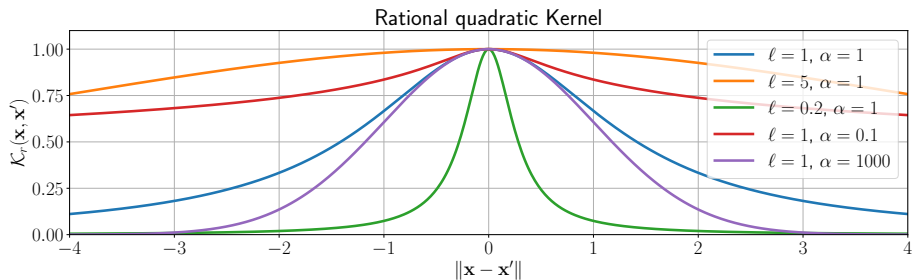
$$\mathcal{K}_g(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2l^2}\right), \quad l : \text{bandwidth}$$



Popular Mercer Kernel [2]

Rational quadratic kernel

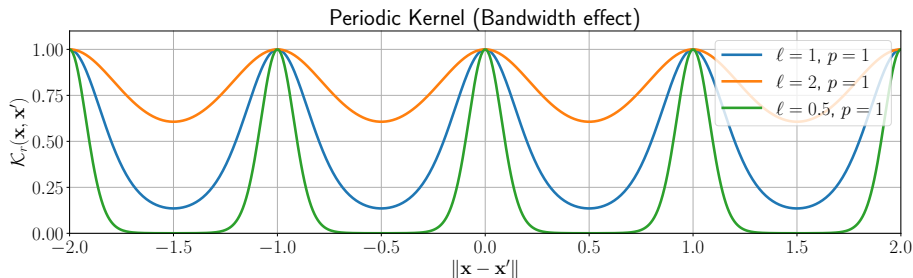
$$\mathcal{K}_r(\mathbf{x}, \mathbf{x}') = \left(1 + \frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\alpha\ell^2}\right)^{-\alpha}, \quad \begin{cases} \ell : \text{bandwidth} \\ \alpha : \text{scale} \end{cases}$$



Popular Mercer Kernel [2]

Periodic kernel

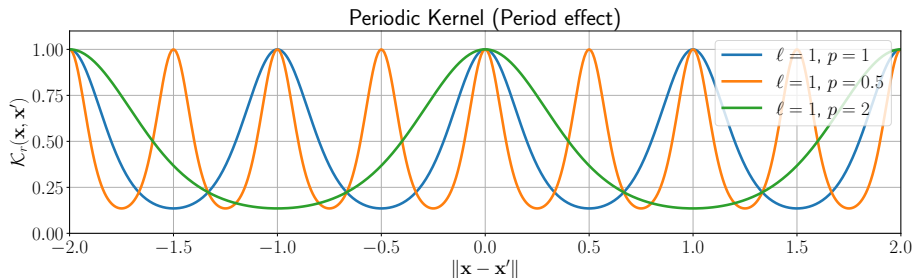
$$\mathcal{K}_p(\mathbf{x}, \mathbf{x}') = \exp \left(-\frac{2}{l^2} \sin^2 \left(\pi \frac{\|\mathbf{x} - \mathbf{x}'\|}{p} \right) \right), \quad \begin{cases} l : \text{bandwidth} \\ \alpha : \text{period} \end{cases}$$



Popular Mercer Kernel [2]

Periodic kernel

$$\mathcal{K}_p(\mathbf{x}, \mathbf{x}') = \exp \left(-\frac{2}{l^2} \sin^2 \left(\pi \frac{\|\mathbf{x} - \mathbf{x}'\|}{p} \right) \right), \quad \begin{cases} l : \text{bandwidth} \\ \alpha : \text{period} \end{cases}$$



Section 3

Gaussian Processes

Gaussian Processes

Gaussian processes (GP) are an approach to defining distribution over functions of the form $f : \mathcal{X} \rightarrow \mathbb{R}$. To this end, we assume $\mathbf{f} = [f(\mathbf{x}_1), \dots, f(\mathbf{x}_M)]^T$ to be jointly Gaussian for any $M > 0$ and with:

- $\boldsymbol{\mu} = [m(\mathbf{x}_1), \dots, m(\mathbf{x}_M)]^T$
- $\Sigma_{ij} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$ or equivalently $\boldsymbol{\Sigma} = \mathbf{K}$

To use the above distribution, we can consider the special case $M = N + 1$ and joint distribution $p([f(\mathbf{x}_1), \dots, f(\mathbf{x}_N), f(\mathbf{x}_q)]^T)$ and infer $f(\mathbf{x}_q)$.

Covariance matrix

A valid covariance matrix must be symmetric and positive semi-definite. These conditions are already met in $\mathbf{K}$ for a Mercer kernel.

Prior Information

We can assume the mean function $m(\cdot)$ and covariance generative kernel $\mathcal{K}(\cdot, \cdot)$ to be the prior information.

Sampling from Prior Information

Sampling from Prior Information

Consider $\mathbf{x} \in \mathbb{R}^D$, $m(\mathbf{x})$ and a Mercer kernel $\mathcal{K}(\cdot, \cdot)$. Then for sampling from the prior we have the following steps:

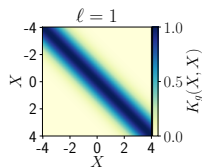
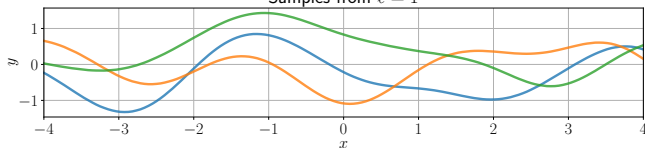
- Assume a set of $\mathbf{x}$ values where we want to evaluate the sample as $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_S\}$.
- Generate the mean vector and covariance matrix as:

$$\boldsymbol{\mu} = \begin{bmatrix} m(\mathbf{x}_1) \\ m(\mathbf{x}_2) \\ \vdots \\ m(\mathbf{x}_S) \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} \mathcal{K}(\mathbf{x}_1, \mathbf{x}_1) & \dots & \mathcal{K}(\mathbf{x}_1, \mathbf{x}_S) \\ \vdots & \ddots & \vdots \\ \mathcal{K}(\mathbf{x}_S, \mathbf{x}_1) & \dots & \mathcal{K}(\mathbf{x}_S, \mathbf{x}_S) \end{bmatrix}$$

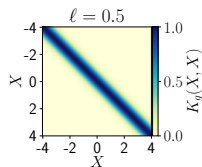
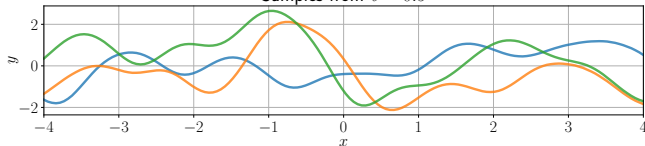
- Sample vector $\mathbf{y} = [y_1, \dots, y_S] \in \mathbb{R}^S$
- The realization of the function at evaluation points is $\{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_S, y_S)\}$

Sampling from Zero Mean and Gaussian Kernel [2]

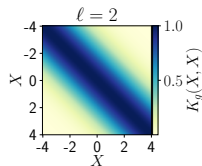
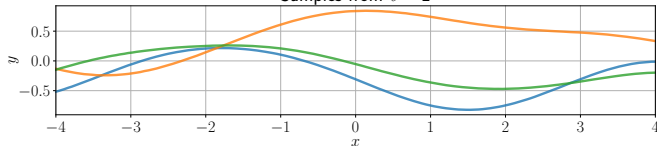
Samples from $\ell = 1$



Samples from $\ell = 0.5$

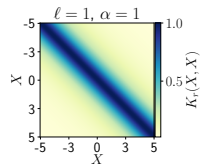
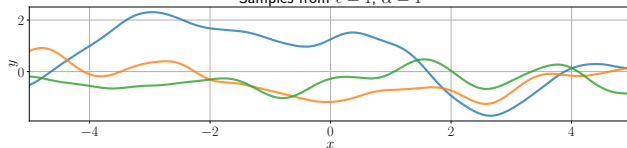


Samples from $\ell = 2$

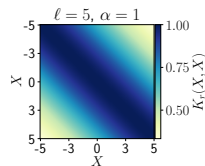
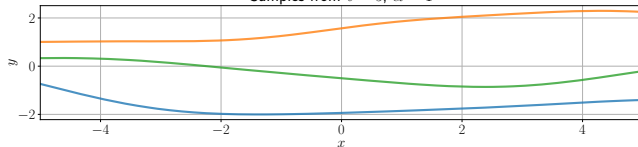


Sampling from Zero Mean and Rationale Kernel [2]

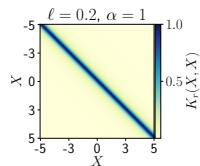
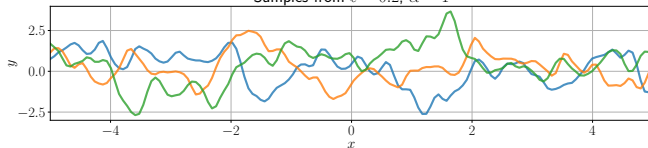
Samples from $\ell = 1, \alpha = 1$



Samples from $\ell = 5, \alpha = 1$

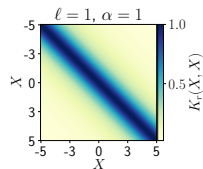
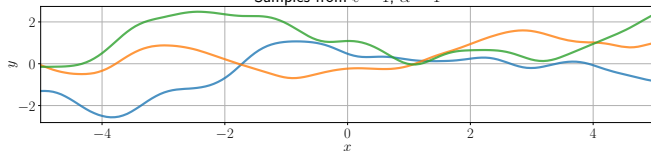


Samples from $\ell = 0.2, \alpha = 1$

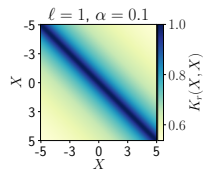
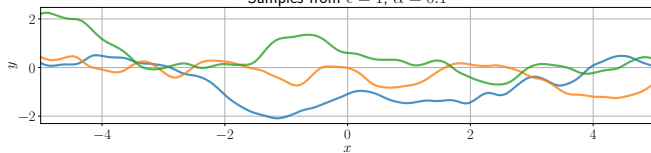


Sampling from Zero Mean and Rationale Kernel [2]

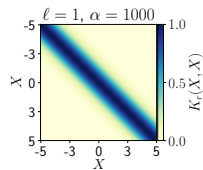
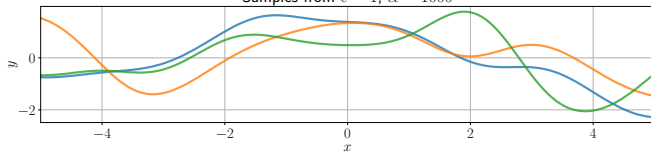
Samples from $\ell = 1, \alpha = 1$



Samples from $\ell = 1, \alpha = 0.1$

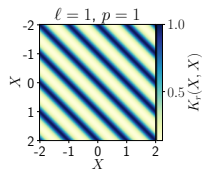
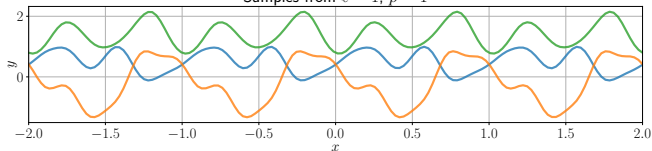


Samples from $\ell = 1, \alpha = 1000$

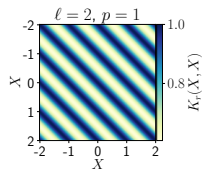
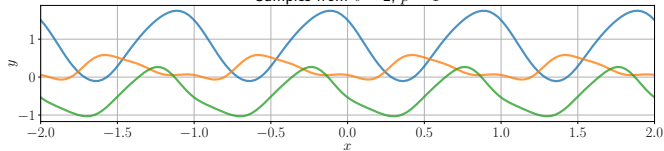


Sampling from Zero Mean and Periodic Kernel [2]

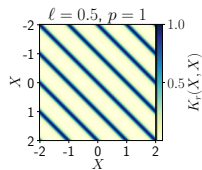
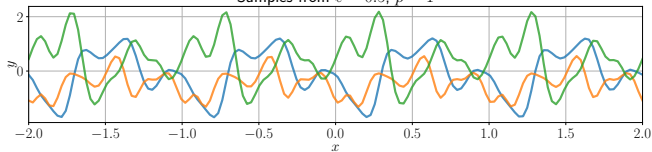
Samples from $\ell = 1, p = 1$



Samples from $\ell = 2, p = 1$

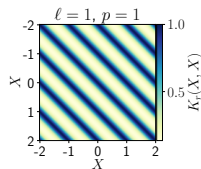
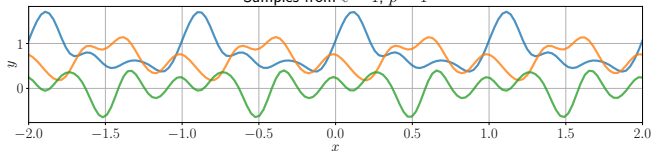


Samples from $\ell = 0.5, p = 1$

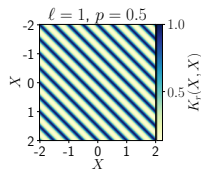
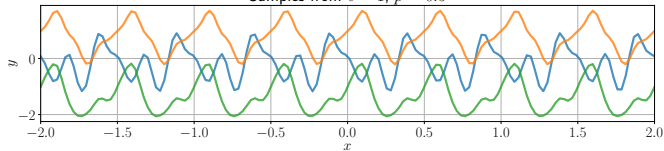


Sampling from Zero Mean and Periodic Kernel [2]

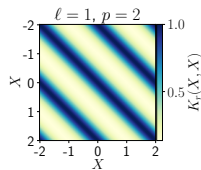
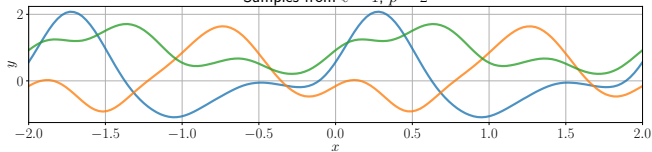
Samples from $\ell = 1, p = 1$



Samples from $\ell = 1, p = 0.5$



Samples from $\ell = 1, p = 2$



Posterior for Noise Free Observations

Noise Free Observations

Suppose we observe training dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ where $y_n = f(\mathbf{x}_n)$ is noise-free observation and we have queries $\{\mathbf{x}_i^{(e)}\}_{i=1}^{N_\star}$. Assume:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{X}_\star = \begin{bmatrix} \mathbf{x}_1^{(e)T} \\ \vdots \\ \mathbf{x}_{N_\star}^{(e)T} \end{bmatrix}, \quad \boldsymbol{\mu}_\mathbf{X} = \begin{bmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_N) \end{bmatrix}, \quad \boldsymbol{\mu}_\star = \begin{bmatrix} m(\mathbf{x}_1^{(e)}) \\ \vdots \\ m(\mathbf{x}_{N_\star}^{(e)}) \end{bmatrix}$$
$$\mathbf{f}_\mathbf{X} = \begin{bmatrix} f(\mathbf{x}_1) \\ \vdots \\ f(\mathbf{x}_N) \end{bmatrix}, \quad \mathbf{f}_\star = \begin{bmatrix} f(\mathbf{x}_1^{(e)}) \\ \vdots \\ f(\mathbf{x}_{N_\star}^{(e)}) \end{bmatrix}, \quad \begin{cases} \mathbf{K}_{\mathbf{X},\mathbf{X}} = \mathcal{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{N \times N} \\ \mathbf{K}_{\mathbf{X},\star} = \mathcal{K}(\mathbf{X}, \mathbf{X}_\star) \in \mathbb{R}^{N \times N_\star} \\ \mathbf{K}_{\star,\star} = \mathcal{K}(\mathbf{X}_\star, \mathbf{X}_\star) \in \mathbb{R}^{N_\star \times N_\star} \end{cases}$$

Prior Information

Prior Based on GP

Based on GP, we have joint distribution $p(\mathbf{f}_X, \mathbf{f}_\star | \mathbf{X}, \mathbf{X}_\star)$ as:

$$\begin{bmatrix} \mathbf{f}_X \\ \mathbf{f}_\star \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}_x \\ \boldsymbol{\mu}_\star \end{bmatrix}, \begin{bmatrix} \mathbf{K}_{X,X} & \mathbf{K}_{X,\star} \\ \mathbf{K}_{X,\star}^T & \mathbf{K}_{\star,\star} \end{bmatrix} \right)$$

Posterior By MVN Conditionals

Assuming we have observe $\mathbf{f}_X$, we can compute the posterior over $\mathbf{f}_\star$ using MVN conditionals.

MVN Conditionals

Suppose $\mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix}$ is jointly Gaussian with $\boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}$, $\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{bmatrix}$. Then the posterior conditional is given by $p(\mathbf{y}_1 | \mathbf{y}_2) = \mathcal{N}(\mathbf{y}_1 | \boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2})$ where:

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} (\mathbf{y}_2 - \boldsymbol{\mu}_2), \quad \boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12} \boldsymbol{\Sigma}_{22}^{-1} \boldsymbol{\Sigma}_{21}$$

Posterior for Noise Free Observations

Calculating Posterior

Using prior distribution and MVN conditionals, we conclude $p(\mathbf{f}_\star | \mathcal{D}, \mathbf{X}_\star) = \mathcal{N}(\mathbf{f}_\star | \boldsymbol{\mu}_\star^{post}, \boldsymbol{\Sigma}_\star^{post})$ where:

$$\begin{aligned}\boldsymbol{\mu}_\star^{post} &= \boldsymbol{\mu}_\star + \mathbf{K}_{X,\star}^T \mathbf{K}_{X,X}^{-1} (\mathbf{f}_X - \boldsymbol{\mu}_X) \\ \boldsymbol{\Sigma}_\star^{post} &= \mathbf{K}_{\star,\star} - \mathbf{K}_{X,\star}^T \mathbf{K}_{X,X}^{-1} \mathbf{K}_{X,\star}\end{aligned}$$

Training Data Interpolator

Suppose we use zero mean prior ($m(\mathbf{x}) = 0$) and kernel $\mathcal{K}(\cdot, \cdot)$. Assume the following case:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{X}_\star = [\mathbf{x}_i^T]$$

Then, using equation $\mathbf{K}_{X,\star} = \mathbf{K}_{X,X}(:, i)$, we can show that:

$$\mathbf{K}_{X,\star}^T \mathbf{K}_{X,X}^{-1} = \mathbf{K}_{X,X}(i, :) \mathbf{K}_{X,X}^{-1} = \mathbf{e}_i^T$$

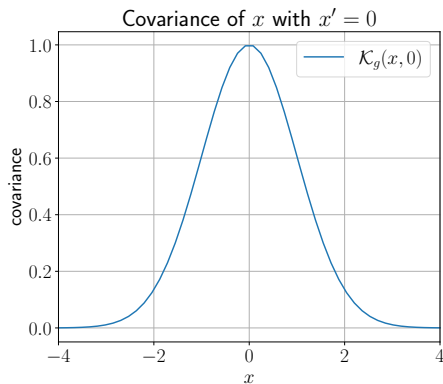
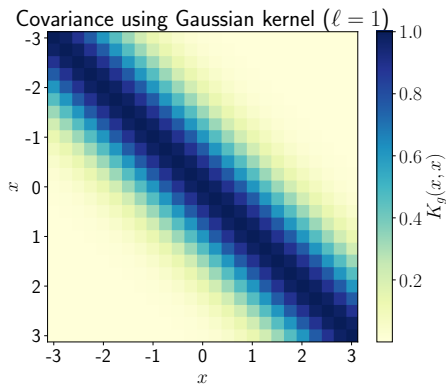
Using above equality, we have:

$$\begin{aligned} \mu_\star^{post} &= \mu_\star + \mathbf{K}_{X,\star}^T \mathbf{K}_{X,X}^{-1} (\mathbf{f}_X - \mu_X) = 0 + \mathbf{e}_i^T (\mathbf{f}_X - \mathbf{0}) = \mathbf{f}_X(i) = f(\mathbf{x}_i) \\ \Sigma_\star^{post} &= \mathbf{K}_{\star,\star} - \mathbf{K}_{X,\star}^T \mathbf{K}_{X,X}^{-1} \mathbf{K}_{X,\star} = \mathcal{K}(\mathbf{x}_i, \mathbf{x}_i) - \mathbf{e}_i^T \mathbf{K}_{X,\star} \\ &= \mathcal{K}(\mathbf{x}_i, \mathbf{x}_i) - \mathcal{K}(\mathbf{x}_i, \mathbf{x}_i) = 0 \end{aligned}$$

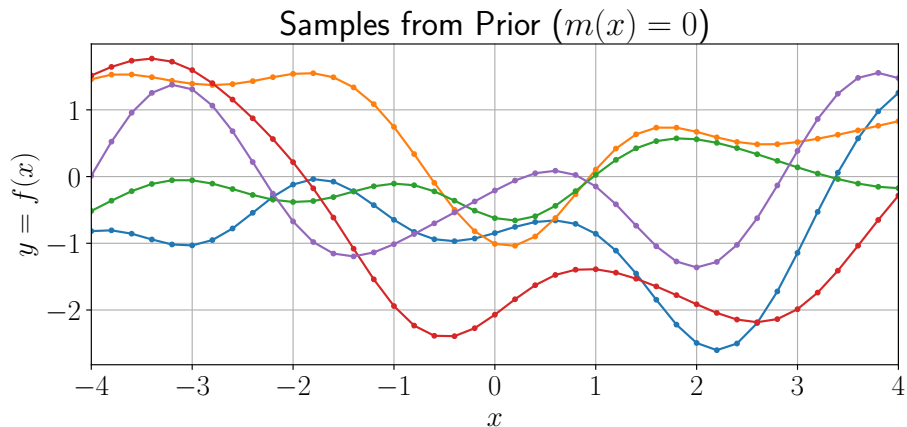
Example: Kernel Introduction [3]

Gaussian Kernel

Assume Gaussian kernel with unit bandwidth as: $\mathcal{K}_g(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2}\right)$

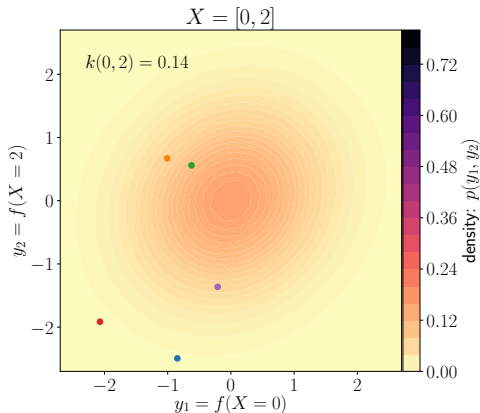
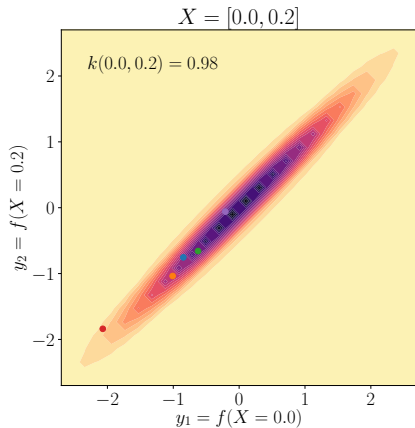


Example: Prior Samples [3]

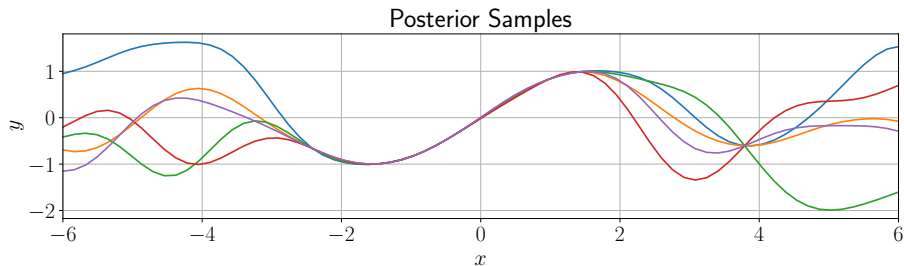
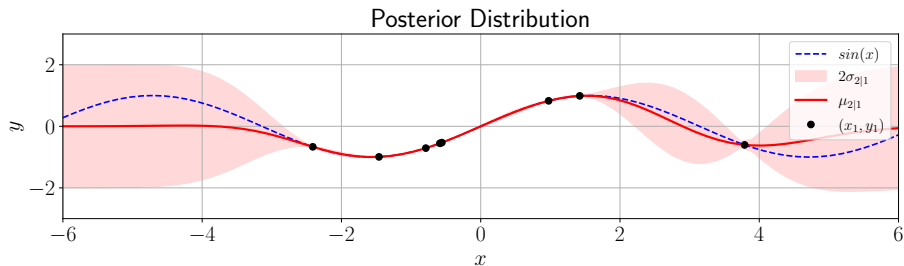


Example: Prior Low Dimensional Samples [3]

2D marginal: $y \sim \mathcal{N}(0, k(X, X))$



Example: Posterior Distribution [3]



Posterior for Noisy Observations

Noisy Observations

Suppose we observe training dataset $\mathcal{D} = \{(\mathbf{x}_n, y_n)\}_{n=1}^N$ where $y_n = f(\mathbf{x}_n) + \epsilon_n$ ($\epsilon_n \sim \mathcal{N}(0, \sigma_y^2)$) is noisy observation and we have queries $\{\mathbf{x}_i^{(e)}\}_{i=1}^{N_\star}$. Assume:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{X}_\star = \begin{bmatrix} \mathbf{x}_1^{(e)T} \\ \vdots \\ \mathbf{x}_{N_\star}^{(e)T} \end{bmatrix}, \quad \boldsymbol{\mu}_\mathbf{X} = \begin{bmatrix} m(\mathbf{x}_1) \\ \vdots \\ m(\mathbf{x}_N) \end{bmatrix}, \quad \boldsymbol{\mu}_\star = \begin{bmatrix} m(\mathbf{x}_1^{(e)}) \\ \vdots \\ m(\mathbf{x}_{N_\star}^{(e)}) \end{bmatrix}$$
$$\mathbf{y} = \begin{bmatrix} f(\mathbf{x}_1) + \epsilon_1 \\ \vdots \\ f(\mathbf{x}_N) + \epsilon_N \end{bmatrix}, \quad \mathbf{f}_\star = \begin{bmatrix} f(\mathbf{x}_1^{(e)}) \\ \vdots \\ f(\mathbf{x}_{N_\star}^{(e)}) \end{bmatrix}, \quad \begin{cases} \mathbf{K}_{\mathbf{X},\mathbf{X}} = \mathcal{K}(\mathbf{X}, \mathbf{X}) \in \mathbb{R}^{N \times N} \\ \mathbf{K}_{\mathbf{X},\star} = \mathcal{K}(\mathbf{X}, \mathbf{X}_\star) \in \mathbb{R}^{N \times N_\star} \\ \mathbf{K}_{\star,\star} = \mathcal{K}(\mathbf{X}_\star, \mathbf{X}_\star) \in \mathbb{R}^{N_\star \times N_\star} \end{cases}$$

Prior Information

Prior Based on GP

In this case, the covariance of observed noisy responses is:

$$\begin{aligned}\text{Cov}[y_i, y_j] &= \text{Cov}[f(\mathbf{x}_i) + \epsilon_i, f(\mathbf{x}_j) + \epsilon_j] = \text{Cov}[f(\mathbf{x}_i), f(\mathbf{x}_j)] + \text{Cov}[\epsilon_i, \epsilon_j] \\ &= \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) + \sigma_y^2 \delta_{ij}\end{aligned}$$

Thus we conclude:

$$\text{Cov}[\mathbf{y}|\mathbf{X}] = \mathbf{K}_{X,X} + \sigma_y^2 \mathbf{I} \triangleq \mathbf{K}_\sigma$$

Based on GP, we have joint distribution $p(\mathbf{f}_X, \mathbf{f}_\star | \mathbf{X}, \mathbf{X}_\star)$ as:

$$\begin{bmatrix} \mathbf{y} \\ \mathbf{f}_\star \end{bmatrix} \sim \mathcal{N} \left(\begin{bmatrix} \boldsymbol{\mu}_x \\ \boldsymbol{\mu}_\star \end{bmatrix}, \begin{bmatrix} \mathbf{K}_\sigma & \mathbf{K}_{X,\star} \\ \mathbf{K}_{X,\star}^T & \mathbf{K}_{\star,\star} \end{bmatrix} \right)$$

Posterior By MVN Conditionals

Assuming we have observe $\mathbf{f}_X$, we can compute the posterior over $\mathbf{f}_\star$ using MVN conditionals.

Posterior for Noise Free Observations

MVN Conditionals

Suppose $\mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix}$ is jointly Gaussian with $\boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}$, $\boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{bmatrix}$ Then the posterior conditional is given by $p(\mathbf{y}_1|\mathbf{y}_2) = \mathcal{N}(\mathbf{y}_1|\boldsymbol{\mu}_{1|2}, \boldsymbol{\Sigma}_{1|2})$ where:

$$\boldsymbol{\mu}_{1|2} = \boldsymbol{\mu}_1 + \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}(\mathbf{y}_2 - \boldsymbol{\mu}_2), \quad \boldsymbol{\Sigma}_{1|2} = \boldsymbol{\Sigma}_{11} - \boldsymbol{\Sigma}_{12}\boldsymbol{\Sigma}_{22}^{-1}\boldsymbol{\Sigma}_{21}$$

Calculating Posterior

Using prior distribution and MVN conditionals, we conclude $p(\mathbf{f}_*|\mathcal{D}, \mathbf{X}_*) = \mathcal{N}(\mathbf{f}_*|\boldsymbol{\mu}_*^{post}, \boldsymbol{\Sigma}_*^{post})$ where:

$$\begin{aligned}\boldsymbol{\mu}_*^{post} &= \boldsymbol{\mu}_* + \mathbf{K}_{X,*}^T \mathbf{K}_\sigma^{-1}(\mathbf{y} - \boldsymbol{\mu}_X) \\ \boldsymbol{\Sigma}_*^{post} &= \mathbf{K}_{*,*} - \mathbf{K}_{X,*}^T \mathbf{K}_\sigma^{-1} \mathbf{K}_{X,*}\end{aligned}$$

Single Test Input

Single Test Input

Suppose we use zero mean prior ($m(\mathbf{x}) = 0$) and kernel $\mathcal{K}(\cdot, \cdot)$. Assume the following case:

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{X}_* = [\mathbf{x}_*^T]$$

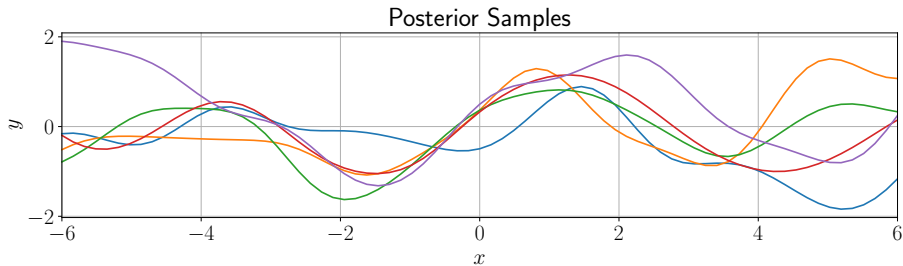
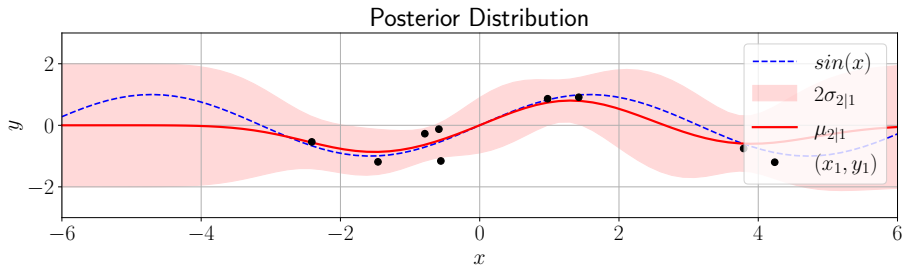
Then we can show that $p(f_* | \mathcal{D}, \mathbf{x}_*) = \mathcal{N}(f_* | 0 + \mathbf{k}_*^T \mathbf{K}_\sigma^{-1}(\mathbf{y} - \mathbf{0}), k_{**} - \mathbf{k}_*^T \mathbf{K}_\sigma^{-1} \mathbf{k}_*)$, where:

$$\mathbf{k}_* = \begin{bmatrix} \mathcal{K}(\mathbf{x}_*, \mathbf{x}_1) \\ \vdots \\ \mathcal{K}(\mathbf{x}_*, \mathbf{x}_N) \end{bmatrix}, \quad k_{**} = \mathcal{K}(\mathbf{x}_*, \mathbf{x}_*)$$

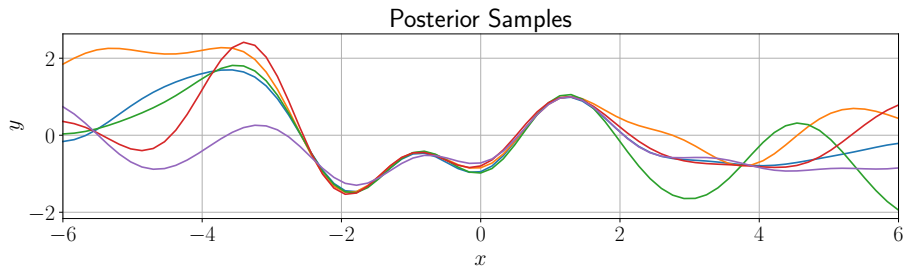
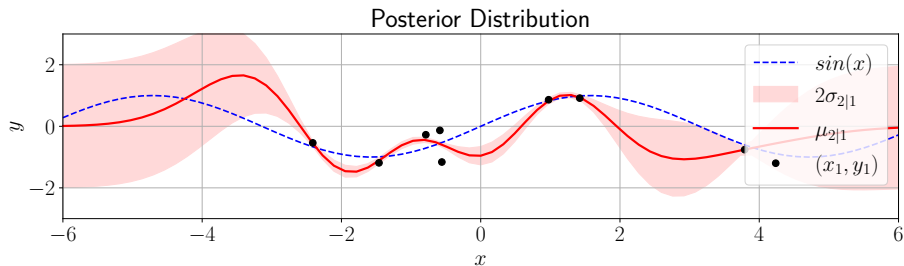
Thus we have conditional mean as:

$$\mu_{*|X} = \mathbf{k}_*^T (\mathbf{K}_\sigma^{-1} \mathbf{y}) \triangleq \mathbf{k}_*^T \boldsymbol{\alpha} = \sum_{n=1}^N \mathcal{K}(\mathbf{x}_*, \mathbf{x}_n) \alpha_n$$

Example: Posterior Distribution ($\sigma_y = 0.5$) [3]



Example: Posterior Distribution ($\sigma_y = 0.05$) [3]



Section 4

Support Vector Machines

Subsection 1

Hard Margin

Approach Definition

Support vector machines (SVMs) are non-probabilistic models for classification and regression formulated as:

$$f(\mathbf{x}) = \sum_{n=1}^N \alpha_n \mathcal{K}(\mathbf{x}, \mathbf{x}_n)$$

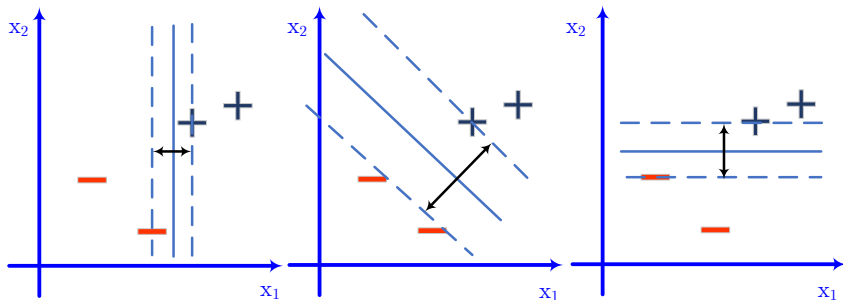
Idea Behind SVMs

- When N is large, kernel methods are not efficient.
- SVMs solve the aforementioned deficiency by ensuring that many of α_i coefficients are zero.
- *Support vectors* are training samples $\mathbf{x}_i$ whose corresponding coefficient α_i are not zero.

Widest Street Concept [4]

Hard Margin

In SVM with hard margin, we assume a two class classification problem where the training samples are linearly separable. We are looking for the widest classification margin.

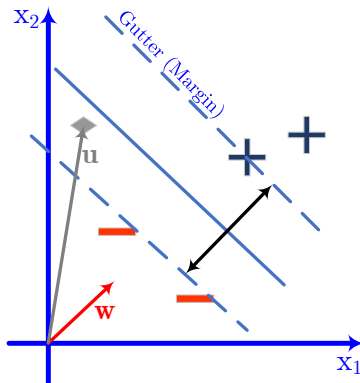


Decision Rule [4]

Intuition

Assume $\mathbf{w}$ to be perpendicular line to the decision street. As the projection of $\mathbf{u}$ on $\mathbf{w}$ increases, the unknown samples tends to lie on the + side. Thus:

$$\begin{cases} + & \text{if } \langle \mathbf{w}, \mathbf{x} \rangle + w_0 \geq 0 \\ - & \text{if } \langle \mathbf{w}, \mathbf{x} \rangle + w_0 < 0 \end{cases}$$



Toward Constraining

Considering a Margin

Assume samples $\mathbf{x}_+$ and $\mathbf{x}_-$ for positive and negative samples, respectively. Then we impose the following inequalities:

$$\langle \mathbf{w}, \mathbf{x}_+ \rangle + w_0 \geq 1$$

$$\langle \mathbf{w}, \mathbf{x}_- \rangle + w_0 \leq 1$$

We can encode the label using definition of y as:

$$y \triangleq \begin{cases} +1 & + \text{ samples} \\ -1 & - \text{ samples} \end{cases}$$

Using the definition of we can write the enequalities for positive and negative samples as:

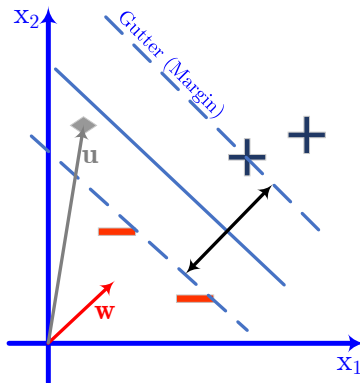
$$\left. \begin{array}{l} y_+ (\langle \mathbf{w}, \mathbf{x}_+ \rangle + w_0) \geq 1 \\ y_- (\langle \mathbf{w}, \mathbf{x}_- \rangle + w_0) \geq 1 \end{array} \right\} \Rightarrow y (\langle \mathbf{w}, \mathbf{x} \rangle + w_0) - 1 \geq 0$$

Samples on the Gutter

Samples on the Gutter

For samples on the gutter (margin), the inequality is reduced to equality as:

$$y(\langle \mathbf{w}, \mathbf{x} \rangle + w_0) - 1 = 0$$

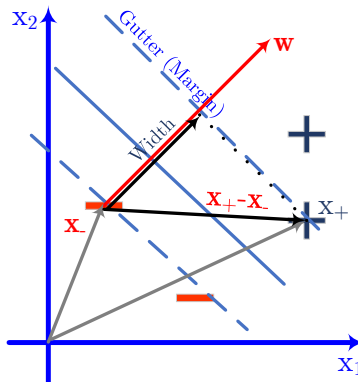


Width of the Street

Width of the Street

The width of the street can be calculated using projection of difference vector $\mathbf{x}_+ - \mathbf{x}_-$ onto normalized normal vector $\mathbf{w}$ as:

$$\text{Width} = \left\langle \mathbf{x}_+ - \mathbf{x}_-, \frac{\mathbf{w}}{\|\mathbf{w}\|} \right\rangle$$



Equal Formulation

For positive samples on the gutter, we have:

$$y_+ (\langle \mathbf{w}, \mathbf{x}_+ \rangle + w_0) = 1 \Rightarrow \langle \mathbf{w}, \mathbf{x}_+ \rangle = 1 - w_0$$

For negative samples on the gutter, we have:

$$y_- (\langle \mathbf{w}, \mathbf{x}_- \rangle + w_0) = 1 \Rightarrow \langle \mathbf{w}, \mathbf{x}_- \rangle = -1 - w_0$$

Using the above equation, we conclude:

$$\begin{aligned} \text{Width} &= \left\langle \mathbf{x}_+ - \mathbf{x}_-, \frac{\mathbf{w}}{\|\mathbf{w}\|} \right\rangle = \frac{1}{\|\mathbf{w}\|} (\langle \mathbf{x}_+, \mathbf{w} \rangle - \langle \mathbf{x}_-, \mathbf{w} \rangle) \\ &= \frac{1}{\|\mathbf{w}\|} (1 - w_0 + 1 + w_0) = \frac{2}{\|\mathbf{w}\|} \end{aligned}$$

Maximizing the Width

Optimization Problem

To maximize the width of street, we have to solve the following optimization problem:

$$\begin{aligned}\hat{\mathbf{w}} &= \operatorname{argmax}_{\mathbf{w}} \frac{2}{\|\mathbf{w}\|} \text{ subject to } y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 \geq 0, i = 1, \dots, N \\ &= \operatorname{argmin}_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 \text{ subject to } y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 \geq 0, i = 1, \dots, N\end{aligned}$$

Lagrangian

The Lagrangian for the above optimization problem is:

$$L = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1]$$

Differentiating the Lagrangian

With Respect to $\mathbf{w}$

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow \hat{\mathbf{w}} = \sum_i \alpha_i y_i \mathbf{x}_i$$

Thus $\mathbf{w}$ is linear combination of some of training samples $\mathbf{x}_i$

With Respect to w_0

$$\frac{\partial L}{\partial w_0} = - \sum_i \alpha_i y_i = 0 \Rightarrow \sum_i \alpha_i y_i = 0$$

Re-writing the Lagrangian

Re-writing the Lagrangian

Plug-in the equation found by differentiating with respect to $\mathbf{w}$ and w_0 into Lagrangian, we have:

$$\begin{aligned} L &= \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y_i (\langle \mathbf{w}, \mathbf{w}_i \rangle + w_0) - 1] \\ &= \frac{1}{2} \left\langle \sum_i \alpha_i y_i \mathbf{x}_i, \sum_j \alpha_j y_j \mathbf{x}_j \right\rangle - \sum_i \alpha_i y_i \left\langle \sum_j \alpha_j y_j \mathbf{x}_j, \mathbf{x}_i \right\rangle - w_0 \overbrace{\sum_i \alpha_i y_i}^{=0} + \sum_i \alpha_i \\ &= \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle, \quad \sum_i \alpha_i y_i = 0 \end{aligned}$$

Lagrangian Form

As we can see, in the resulting Lagrangian, it's dependent upon input samples $\mathbf{x}_i$ is of the inner product form. Thus we just need to know the inner product of input samples for optimization.

Updating the Decision Rule

Updating the Decision Rule

Using equality $\mathbf{w} = \sum_i \alpha_i y_i \mathbf{x}_i$, we can write the decision rule as:

$$\begin{cases} +1 & \text{if } \sum_i \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + w_0 \geq 0 \\ -1 & \text{if } \sum_i \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + w_0 < 0 \end{cases}$$

Deciding on New Sample $\mathbf{x}$

As we can see, in the resulting decision rule, the dependency upon input query sample $\mathbf{x}$ and training input sample $\{\mathbf{x}_i\}_{i=1}^N$ is of the inner product form. Thus we just need to know the inner product of input samples and query sample to decide on the label.

Looking Back into Lagrangian

Dual Form

As we can see, the Lagrangian is:

$$L = \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle, \quad \sum_i \alpha_i y_i = 0$$

The above form is *dual form* of the objective function and the vector of Lagrange multipliers defined as $\boldsymbol{\alpha} = (\alpha_1, \dots, \alpha_N)$ can be found as:

$$\hat{\boldsymbol{\alpha}} = \underset{\boldsymbol{\alpha}}{\operatorname{argmax}} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \text{ subject to } \begin{cases} \sum_i \alpha_i y_i = 0 \\ \alpha_i \geq 0, i = 1, \dots, N \end{cases}$$

Optimization Problem

The optimization problem for widest street can be formulated as:

$$= \underset{\mathbf{w}}{\operatorname{argmin}} \frac{1}{2} \|\mathbf{w}\|^2 \text{ subject to } y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 \geq 0, i = 1, \dots, N$$

The above optimization problem is convex and we see the Lagrangian as:

$$L = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1]$$

Based on KKT conditions, we have:

$$\alpha_i \geq 0, \quad i = 1, \dots, N$$

$$y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) - 1 \geq 0$$

$$\alpha_i (y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) - 1) = 0$$

Complementary Slackness

Based on *complementary slackness*, we have:

$$\begin{cases} y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) - 1 = 0 \\ \text{or} \\ \alpha_i = 0 \end{cases}$$

Remember that the decision is made based on:

$$\begin{cases} +1 & \text{if } \sum_i \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + w_0 \geq 0 \\ -1 & \text{if } \sum_i \alpha_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + w_0 < 0 \end{cases}$$

Thus for each training input sample $\mathbf{x}_i$, either of the following condition may happend:

- $\alpha_i = 0$ and the sample is ignored in the decision.
- $y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) = 1$ the sample is on the gutter.

Support Vectors

The input training samples lie on the gutter are known as support vectors. These samples contribute to the decision rule.

Calculating Parameters

Calculating Parameters

As we see, the dual problem is defined as:

$$\hat{\alpha} = \operatorname{argmax}_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \text{ subject to } \begin{cases} \sum_i \alpha_i y_i = 0 \\ \alpha_i \geq 0, i = 1, \dots, N \end{cases}$$

Now consider the following definitions:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \alpha = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \widehat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{S} = \widehat{\mathbf{X}} \widehat{\mathbf{X}}^T$$

Then we can rewrite the optimization as:

$$\max_{\alpha} \mathbf{1}^T \alpha - \frac{1}{2} \alpha \mathbf{S} \alpha \text{ subject to } \begin{cases} \mathbf{y}^T \alpha = 0 \\ \alpha \geq 0 \end{cases}$$

The above problem can be solved using packages for standard quadratic programming (QP) solvers.

Calculating Parameters

After calculating $\hat{\alpha}$ using QP, we can find model parameters as:

- $\hat{\mathbf{w}} = \sum_i \hat{\alpha}_i y_i \mathbf{x}_i$
- $\hat{w}_0$: Assume $\mathbf{x}_i$ to be a support vector, then we can calculate $\hat{w}_0$ as:

$$y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) = 1 \xrightarrow{\times y_i} \hat{w}_0 = y_i - \langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle$$

In practice we average the value of $\hat{w}_0$ resulting from all support vectors in set $\mathcal{S}$ as:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} (y_i - \langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle) = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(y_i - \sum_{j \in \mathcal{S}} \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle \right)$$

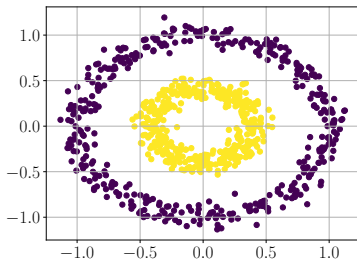
Subsection 2

Hard Margin SVM with Kernel

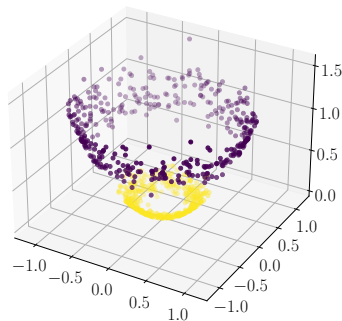
Linear Separability By Increasing Dimensions

Separability by Mapping

$$\phi\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right) = \begin{bmatrix} x_1 \\ x_2 \\ x_1^2 + x_2^2 \end{bmatrix} \Rightarrow \mathcal{K}\left(\overbrace{\begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}}^{x_i}, \overbrace{\begin{bmatrix} x_{j1} \\ x_{j2} \end{bmatrix}}^{x_j}\right) = x_{i1}x_{j1} + x_{i2}x_{j2} + (x_{i1}^2 + x_{i2}^2)(x_{j1}^2 + x_{j2}^2)$$



(a) Original dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$



(b) Mapped dataset $\{(\phi(\mathbf{x}_i), y_i)\}_{i=1}^N$

A Closer Look to Hard Margin SVM

How to Solve Widest Margin Problem

Using QP, we solve:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \quad \mathbf{1}^T \alpha - \frac{1}{2} \alpha^T S \alpha \quad \text{subject to} \quad \begin{cases} \mathbf{y}^T \alpha = 0 \\ \alpha \geq 0 \end{cases}$$

where:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \alpha = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \widehat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$
$$\mathbf{S} = \widehat{\mathbf{X}} \widehat{\mathbf{X}}^T \Rightarrow \begin{cases} \mathbf{S} \in \mathbb{R}^{N \times N} \\ [\mathbf{S}]_{ij} = y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \end{cases}$$

To evaluate the class for a new data, we have:

$$\sum_i \hat{\alpha}_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + \hat{w}_0 \begin{cases} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{cases}$$

Replacing Inner Product with PD Kernel

Hard Margin SVM with Kernel

Assume using a mapping $\phi : \mathbb{R}^D \rightarrow \mathcal{H}$, we convert dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ to $\{(\phi(\mathbf{x}_i), y_i)\}_{i=1}^N$ where $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$. Then we can design hard margin SVM as:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \quad \mathbf{1}^T \alpha - \frac{1}{2} \alpha^T S \alpha \quad \text{subject to} \quad \begin{cases} \mathbf{y}^T \alpha = 0 \\ \alpha \geq 0 \end{cases}$$

where:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \alpha = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \hat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$
$$S = \hat{\mathbf{X}} \hat{\mathbf{X}}^T \Rightarrow \begin{cases} S \in \mathbb{R}^{N \times N} \\ [S]_{ij} = y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = y_i y_j \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) \end{cases}$$

To evaluate the class for a new data, we have:

$$\sum_i \hat{\alpha}_i y_i \underbrace{\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle}_{\mathcal{K}(\mathbf{x}_i, \mathbf{x})} + \hat{w}_0 \begin{cases} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{cases}$$

Intuition

As we see before, for any positive kernel we have:

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

where $\phi : \mathbb{R}^D \rightarrow \mathcal{H}$ and $\mathcal{H}$ is a Hilbert space. The process above can be describes as:

- Mapping input training samples $\{\mathbf{x}_i\}_{i=1}^N$ to Hilbert space $\mathcal{H}$ as $\{\phi(\mathbf{x}_i)\}_{i=1}^N$
- Explicitly finding separating hyper-plane $\mathbf{w} = \sum_i \hat{\alpha}_i y_i \phi(\mathbf{x}_i)$ (implicitly finding $\hat{\alpha}$)
- Finding $\hat{w}_0$ as:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \left(y_i - \sum_{j \in \mathcal{S}} \alpha_j y_j \overbrace{\langle \phi(\mathbf{x}_j), \phi(\mathbf{x}_i) \rangle}^{\mathcal{K}(\mathbf{x}_j, \mathbf{x}_i)} \right)$$

- Evaluating new sample $\mathbf{x}$ as:

$$\sum_i \hat{\alpha}_i y_i \underbrace{\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle}_{\mathcal{K}(\mathbf{x}_i, \mathbf{x})} + \hat{w}_0 \begin{array}{ll} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{array}$$

Subsection 3

Soft Margin SVM

Not Linearly Separable Case

Assume the following case where the training dataset is not linearly separable. Then hard margin SVM cannot be solve due to the infeasibility of constraints.

SVM with Soft Margin

Formulation

Hard margin SVM is formulated as:

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 \quad \text{subject to } y_i (\langle \mathbf{w}, \mathbf{x} \rangle + w_0) \geq 1$$

When the problem is not linearly separable, the set of vectors $\mathbf{w}$ and scalars w_0 that satisfy the constraints is empty.

In soft margin SVM, the above deficiency is solved by updating the problem as:

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i \quad \text{subject to } \begin{cases} y_i (\langle \mathbf{w}, \mathbf{x} \rangle + w_0) \geq 1 - \xi_i \\ \xi_i \geq 0 \end{cases}$$

- $y_i (\langle \mathbf{w}, \mathbf{x} \rangle + w_0) \geq 1 - \xi_i$ allows some samples to enter the margin or even cross the decision hyper-plane
- $C \sum_i \xi_i$ Controls the number of samples that enter the margin or cross the decision hyper-plane

Lagrangian

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i \quad \text{subject to} \quad \begin{cases} y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) \geq 1 - \xi_i \\ \xi_i \geq 0 \end{cases}$$

The Lagrangian for the above problem is:

$$\begin{aligned} L(\mathbf{w}, w_0, \boldsymbol{\alpha}, \boldsymbol{\xi}, \boldsymbol{\mu}) &= \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i - \sum_i \alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 + \xi_i] \\ &\quad - \sum_i \mu_i \xi_i \\ &= \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i - \sum_i \alpha_i y_i \langle \mathbf{w}, \mathbf{x}_i \rangle - \sum_i \alpha_i y_i w_0 \\ &\quad + \sum_i \alpha_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i \end{aligned}$$

Derivatives

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum_i \alpha_i y_i \mathbf{x}_i = 0 \Rightarrow \hat{\mathbf{w}} = \sum_i \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial L}{\partial w_0} \sum_i \alpha_i y_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \alpha_i - \mu_i = 0$$

Updating Lagrangian Formulation

Using above equalities, we can rewrite Lagrangian in terms of dual variables (α and μ) as:

$$\begin{aligned} L = & \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle + C \sum_i \xi_i - \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \\ & + \sum_i \alpha_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i \end{aligned}$$

On the other hand, we know:

$$C \sum_i \xi_i - \sum_i \alpha_i \xi_i - \sum_i \mu_i \xi_i = \sum_i (C - \alpha_i - \mu_i) \xi_i = \sum_i 0 \times \xi_i = 0$$

Dual Formulation

Dual Formulation

Thus we have the following optimization problem:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmax}} -\frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle + \sum_i \alpha_i \quad \text{subject to} \quad \begin{cases} 0 \leq \alpha_i \\ 0 \leq \mu_i \\ \sum_i \alpha_i y_i = 0 \\ C - \alpha_i - \mu_i = 0 \end{cases}$$

The effect of μ_i can be easily translated to upper bounding α_i as:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmax}} -\frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle + \sum_i \alpha_i \quad \text{subject to} \quad \begin{cases} 0 \leq \alpha_i \leq C \\ \sum_i \alpha_i y_i = 0 \end{cases}$$

Thus the only change in comparison to hard margin case, is the upper bound added to α_i .

Intuition

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_i \xi_i \quad \text{subject to} \quad \begin{cases} y_i (\langle \mathbf{w}, \mathbf{x} \rangle + w_0) \geq 1 - \xi_i \\ \xi_i \geq 0 \end{cases}$$

- For large values of C , the feasible interval for α_i is widened and we approach the hard margin SVM.
- For small values of C , the feasible interval for α_i becomes narrower and we allow more samples to cross the margin.

Slackness Complementary

$$\begin{aligned}\alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 + \xi_i] &= 0, \quad i = 1, \dots, N \\ \mu_i \xi_i &= 0, \quad i = 1, \dots, N\end{aligned}$$

Thus:

- $\alpha_i > 0 \Rightarrow \mathbf{x}_i$ is Support Vector $\Rightarrow y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) = 1 - \xi_i$
- $\mu_i > 0 \Rightarrow \begin{cases} \xi_i = 0 \Rightarrow \mathbf{x}_i \text{ is on the margin} \\ C - \alpha_i - \mu_i = 0 \Rightarrow \alpha_i < C \end{cases} \Rightarrow 0 < \alpha_i < C \rightarrow \mathbf{x}_i \text{ is on the margin}$
- $\xi_i > 0 \Rightarrow \begin{cases} \mathbf{x}_i \text{ crosses the margin} \\ \mu_i = 0 \Rightarrow \alpha_i = C \end{cases} \Rightarrow \alpha_i = C \rightarrow \mathbf{x}_i \text{ crosses the margin}$

Slackness Complementary

$$\begin{aligned}\alpha_i [y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) - 1 + \xi_i] &= 0, \quad i = 1, \dots, N \\ \mu_i \xi_i &= 0, \quad i = 1, \dots, N\end{aligned}$$

Thus:

- $y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) > 1 - \xi_i \Rightarrow \mathbf{x}_i$ is NOT Support Vector $\Rightarrow \alpha_i = 0$

$$\left. \begin{array}{l} C - \alpha_i - \mu_i = 0 \\ \alpha_i = 0 \end{array} \right\} \Rightarrow \mu_i = C > 0 \Rightarrow \xi_i = 0 \Rightarrow y_i (\langle \mathbf{w}, \mathbf{x}_i \rangle + w_0) > 1$$

Therefore $\mathbf{x}_i$ is classified correctly and is not on the margin.

Calculating Parameters

Calculating Parameters

Similar to hard SVM, the dual problem is defined as:

$$\hat{\alpha} = \operatorname{argmax}_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_i \sum_j \alpha_i \alpha_j y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \text{ subject to } \begin{cases} \sum_i \alpha_i y_i = 0 \\ 0 \leq \alpha_i \leq C, i = 1, \dots, N \end{cases}$$

Now consider the following definitions:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \boldsymbol{\alpha} = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \widehat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}, \quad \mathbf{S} = \widehat{\mathbf{X}} \widehat{\mathbf{X}}^T$$

Then we can rewrite the optimization as:

$$\min_{\alpha} \mathbf{1}^T \boldsymbol{\alpha} - \frac{1}{2} \boldsymbol{\alpha}^T \mathbf{S} \boldsymbol{\alpha} \text{ subject to } \begin{cases} \mathbf{y}^T \boldsymbol{\alpha} = 0 \\ 0 \leq \boldsymbol{\alpha} \leq C \end{cases}$$

The above problem can be solved using packages for standard quadratic programming (QP) solvers.

Calculating Parameters

Calculating Parameters

After calculating $\hat{\alpha}$ using QP, we can find model parameters as:

- $\hat{\mathbf{w}} = \sum_i \hat{\alpha}_i y_i \mathbf{x}_i$
- $\hat{w}_0$: Assume $\mathbf{x}_i$ to be a support vector *on the margin* ($0 < \alpha_i < C$), then we can calculate $\hat{w}_0$ as:

$$y_i (\langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle + \hat{w}_0) = 1 \xrightarrow{\times y_i} \hat{w}_0 = y_i - \langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle$$

In practice we average the value of $\hat{w}_0$ resulting from all support vectors on the margin in set $\mathcal{S}_m$ as:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} (y_i - \langle \hat{\mathbf{w}}, \mathbf{x}_i \rangle) = \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} \left(y_i - \sum_{j \in \mathcal{S}_m} \alpha_j y_j \langle \mathbf{x}_j, \mathbf{x}_i \rangle \right)$$

Subsection 4

Soft Margin SVM with Kernel

A Closer Look to Soft Margin SVM

How to Solve Widest Margin Problem

Using QP, we solve:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \quad \mathbf{1}^T \alpha - \frac{1}{2} \alpha^T S \alpha \quad \text{subject to} \quad \begin{cases} \mathbf{y}^T \alpha = 0 \\ 0 \leq \alpha \leq C \end{cases}$$

where:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \alpha = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \widehat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$
$$\mathbf{S} = \widehat{\mathbf{X}} \widehat{\mathbf{X}}^T \Rightarrow \begin{cases} \mathbf{S} \in \mathbb{R}^{N \times N} \\ [\mathbf{S}]_{ij} = y_i y_j \langle \mathbf{x}_i, \mathbf{x}_j \rangle \end{cases}$$

To evaluate the class for a new data, we have:

$$\sum_i \hat{\alpha}_i y_i \langle \mathbf{x}_i, \mathbf{x} \rangle + \hat{w}_0 \begin{cases} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{cases}$$

Replacing Inner Product with PD Kernel

Soft Margin SVM with Kernel

Assume using a mapping $\phi : \mathbb{R}^D \rightarrow \mathcal{H}$, we convert dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ to $\{(\phi(\mathbf{x}_i), y_i)\}_{i=1}^N$ where $\mathcal{K}(\mathbf{x}, \mathbf{x}') = \langle \phi(\mathbf{x}), \phi(\mathbf{x}') \rangle$. Then we can design soft margin SVM as:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \quad \mathbf{1}^T \alpha - \frac{1}{2} \alpha^T S \alpha \quad \text{subject to} \quad \begin{cases} \mathbf{y}^T \alpha = 0 \\ 0 \leq \alpha \leq C \end{cases}$$

where:

$$\mathbf{1} = \begin{bmatrix} 1 \\ \vdots \\ 1 \end{bmatrix}_{N \times 1}, \quad \alpha = \begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_N \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} y_1 \\ \vdots \\ y_N \end{bmatrix}, \quad \hat{\mathbf{X}} = \operatorname{diag}(\mathbf{y}) \begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_N^T \end{bmatrix}$$
$$S = \hat{\mathbf{X}} \hat{\mathbf{X}}^T \Rightarrow \begin{cases} S \in \mathbb{R}^{N \times N} \\ [S]_{ij} = y_i y_j \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = y_i y_j \mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) \end{cases}$$

To evaluate the class for a new data, we have:

$$\sum_i \hat{\alpha}_i y_i \underbrace{\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle}_{\mathcal{K}(\mathbf{x}_i, \mathbf{x})} + \hat{w}_0 \begin{cases} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{cases}$$

Intuition

As we see before, for any positive kernel we have:

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle$$

where $\phi : \mathbb{R}^D \rightarrow \mathcal{H}$ and $\mathcal{H}$ is a Hilbert space. The process above can be describes as:

- Mapping input training samples $\{\mathbf{x}_i\}_{i=1}^N$ to Hilbert space $\mathcal{H}$ as $\{\phi(\mathbf{x}_i)\}_{i=1}^N$
- Explicitly finding separating hyper-plane $\mathbf{w} = \sum_i \hat{\alpha}_i y_i \phi(\mathbf{x}_i)$ (implicitly finding $\hat{\alpha}$)
- Finding $\hat{w}_0$ as:

$$\hat{w}_0 = \frac{1}{|\mathcal{S}_m|} \sum_{i \in \mathcal{S}_m} \left(y_i - \sum_{j \in \mathcal{S}_m} \alpha_j y_j \overbrace{\langle \phi(\mathbf{x}_j), \phi(\mathbf{x}_i) \rangle}^{\mathcal{K}(\mathbf{x}_j, \mathbf{x}_i)} \right)$$

- Evaluating new sample $\mathbf{x}$ as:

$$\sum_i \hat{\alpha}_i y_i \underbrace{\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}) \rangle}_{\mathcal{K}(\mathbf{x}_i, \mathbf{x})} + \hat{w}_0 \begin{array}{ll} \geq 0 & \Rightarrow y = +1 \\ < 0 & \Rightarrow y = -1 \end{array}$$

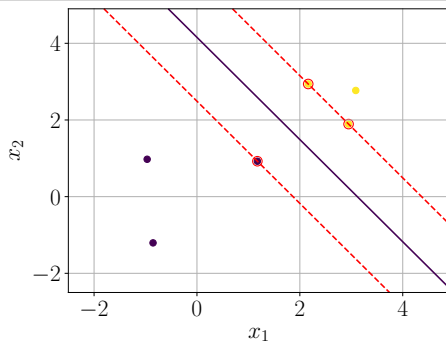
Subsection 5

Samples of SVM

Hard Margin SVM

Hard Margin SVM

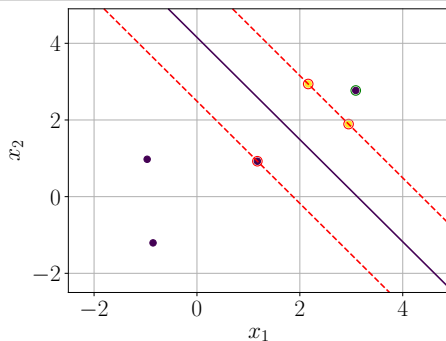
$$\mathbf{X} = \begin{bmatrix} 2.16 & 2.94 \\ 2.95 & 1.89 \\ 3.09 & 2.77 \\ 1.17 & 0.92 \\ -0.97 & 0.98 \\ -0.85 & -1.21 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1.00 \\ 1.00 \\ 1.00 \\ -1.00 \\ -1.00 \\ -1.00 \end{bmatrix}, \Rightarrow \hat{\alpha} = \begin{bmatrix} 0.1105 \\ 0.3898 \\ 0.0000 \\ 0.5003 \\ 0.0000 \\ 0.0000 \end{bmatrix}, \hat{w}_0 = \begin{bmatrix} -2.4943 \end{bmatrix}$$



Soft Margin SVM

Soft Margin SVM with $C = 10$

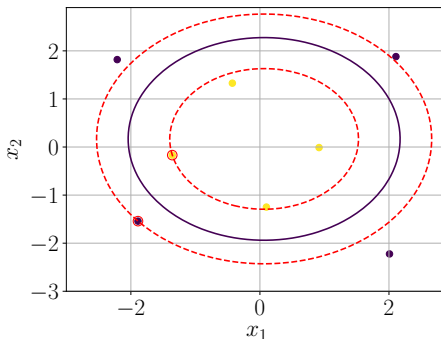
$$\mathbf{X} = \begin{bmatrix} 2.16 & 2.94 \\ 2.95 & 1.89 \\ 3.09 & 2.77 \\ 1.17 & 0.92 \\ -0.97 & 0.98 \\ -0.85 & -1.21 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 1.00 \\ 1.00 \\ -1.00 \\ -1.00 \\ -1.00 \\ -1.00 \end{bmatrix}, \quad \Rightarrow \hat{\alpha} = \begin{bmatrix} 5.5408 \\ 8.1495 \\ 10.0000 \\ 3.6904 \\ 0.0000 \\ 0.0000 \end{bmatrix}, \quad \hat{w}_0 = \begin{bmatrix} -2.4943 \end{bmatrix}$$



Hard Margin kernel-SVM

Hard Margin kernel-SVM

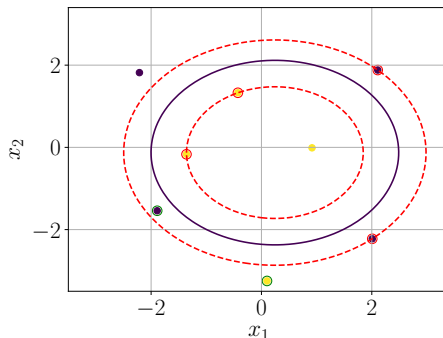
$$\mathbf{X} = \begin{bmatrix} 0.92 & -0.01 \\ -0.43 & 1.33 \\ -1.36 & -0.17 \\ 0.10 & -1.25 \\ -2.21 & 1.82 \\ -1.89 & -1.54 \\ 2.01 & -2.22 \\ 2.11 & 1.88 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1.00 \\ 1.00 \\ 1.00 \\ 1.00 \\ -1.00 \\ -1.00 \\ -1.00 \\ -1.00 \end{bmatrix}, \Rightarrow \hat{\alpha} = \begin{bmatrix} 0.0000 \\ 0.0000 \\ 0.1066 \\ 0.0000 \\ 0.0000 \\ 0.1066 \\ 0.0000 \\ 0.0000 \end{bmatrix}, \hat{w}_0 = \begin{bmatrix} 1.9156 \end{bmatrix}$$



Soft Margin kernel-SVM

Soft Margin kernel-SVM with $C = 10$

$$\mathbf{X} = \begin{bmatrix} 0.92 & -0.01 \\ -0.43 & 1.33 \\ -1.36 & -0.17 \\ 0.10 & -3.25 \\ -2.21 & 1.82 \\ -1.89 & -1.54 \\ 2.01 & -2.22 \\ 2.11 & 1.88 \end{bmatrix}, \mathbf{y} = \begin{bmatrix} 1.00 \\ 1.00 \\ 1.00 \\ 1.00 \\ -1.00 \\ -1.00 \\ -1.00 \\ -1.00 \end{bmatrix}, \Rightarrow \hat{\alpha} = \begin{bmatrix} 0.0000 \\ 2.7729 \\ 3.8414 \\ 10.0000 \\ 0.0000 \\ 10.0000 \\ 6.4263 \\ 0.1879 \end{bmatrix}, \hat{w}_0 = \begin{bmatrix} 2.0089 \end{bmatrix}$$



Subsection 6

Multiclass SVM

One-Versus-Rest Strategy

Approach

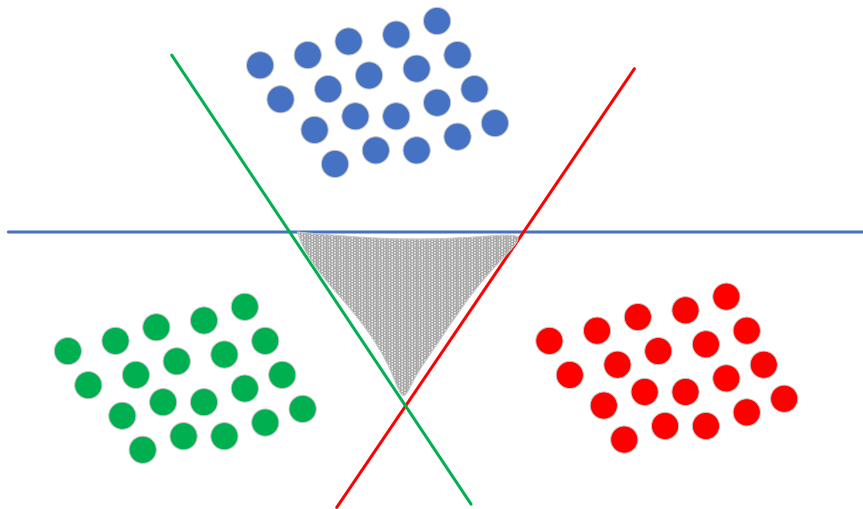
Assume we have C classes labels as $y \in \{1, 2, \dots, C\}$. Then:

- For each value of $c \in \{1, \dots, C\}$, we train a SVM ($f_c(\mathbf{x}) = \mathbf{W}^{(c)}\mathbf{x} + w_0^{(c)}$) over the following dataset:
 - Samples belonging to class c are labeled as $+1$
 - Samples belonging to classes other than c are labeled as -1
- For a new sample $\mathbf{x}$, we find the label as: $\hat{y}(\mathbf{x}) = \operatorname{argmax}_c f_c(\mathbf{x})$

Disadvantages

- Some regions in this method becomes ambiguous (Regions where $f_c(\mathbf{x}) < 0, c = 1, \dots, C$)
- The value of $f_c(\mathbf{x})$ are not calibrated (having $f_1(\mathbf{x}) = l$ and $f_2(\mathbf{x}) = l$ is not informative due to un-calibrated function values)
- Even if the original dataset is balanced across different classes, then the dataset generated to train each $f_i(\mathbf{x})$ is imbalanced.

One-Versus-Rest Strategy



One-Versus-One Strategy

Approach

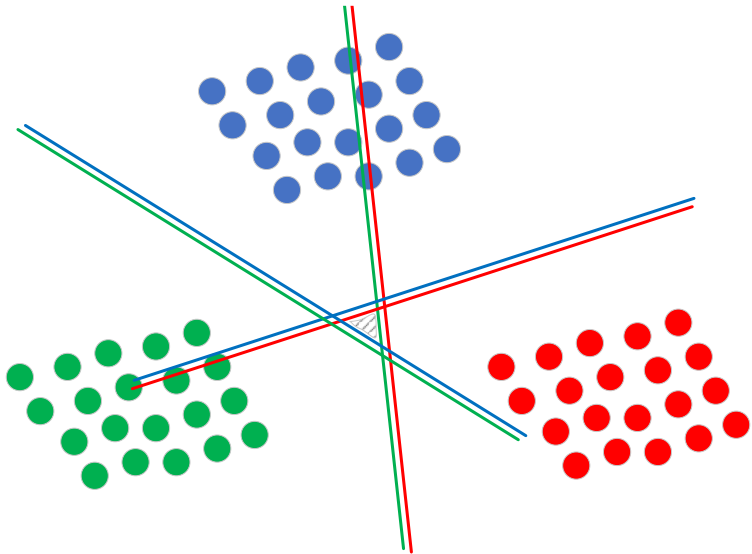
Assume we have C classes labels as $y \in \{1, 2, \dots, C\}$. Then:

- For each possible pair of labels $(c, k) : c, k \in \{1, \dots, C\}$, we train a SVM $(f_{ck}(\mathbf{x}) = \mathbf{W}^{(ck)}\mathbf{x} + w_0^{(ck)})$ over the following dataset:
 - Samples belonging to class c are labeled as $+1$
 - Samples belonging to class k are labeled as -1
- For a new sample $\mathbf{x}$, we find the label as: $\hat{y}(\mathbf{x}) = \text{MaxVote}(\{f_{ck}(\mathbf{x})\})$

Disadvantages

- Some regions in this method becomes ambiguous (Equally voted regions)
- We need to train $\mathcal{O}(C^2)$ models

One-Versus-Rest Strategy



References I



“Kernels and kernel methods (10/09/13),”

https://www.cs.princeton.edu/~bee/courses/scribe/lec_10_09_2013.pdf,

Accessed: 2022-09-24.



“Gaussian processes (3/3) - exploring kernels,”

<https://peterroelants.github.io/posts/gaussian-process-kernels/>,

Accessed: 2022-09-24.



“Gaussian processes (1/3) - from scratch,”

<https://peterroelants.github.io/posts/gaussian-process-tutorial/>,

Accessed: 2022-09-24.



“16. learning: Support vector machines,”

https://www.youtube.com/watch?v=_PwhiWxHK8o&t=1604s,

Accessed: 2022-09-24.